模块踩踏之覆写导出函数绕过 CFG

- **Code Cave 注入 vs Module Stomping**

对比项	Code Cave 注入	Module Stomping
写入位置	.text 节末尾的 padding（0x00/0xCC）	已有导出函数的代码开头
写入目标	无人使用的空地	一个有名字、有地址、CFG 认账的函数
CFG 检查结果	❌ 拒绝（不是合法函数入口）	✅ 放行（是合法函数入口）
副作用	无（没覆盖任何东西）	原函数被破坏，调用它会执行 Shellcode
需要的 API	WriteProcessMemory + CreateRemoteThread	同左，但多了步“查导出函数地址”

一句话总结：Code Cave 是“在空地上盖房”，但 CFG 不给你地契；Module Stomping 是“把别人的房子拆了改建”，因为地契是别人的、合法的，CFG 不查你。

- **目标选择：amsi.dll!AmsiScanBuffer**

我们用 amsi.dll!AmsiScanBuffer 作为靶子，理由：

- amsi.dll 在 PowerShell 进程里必然加载。
- AmsiScanBuffer 是 AMSI 的入口函数，覆写它相当于“打残 AMSI”。
- 它不是高频调用的函数，覆写后短时间内不会引起目标进程异常。

副作用：覆写后，该函数原有的扫描逻辑被 Shellcode 替换。如果目标进程之后调用它，会执行我们的 Shellcode，然后返回。

- **代码示例（见文件夹）**
  - **第一版：明文 API 实现**
  - **第二版：无 IAT 版本（.h 文件见第一周课程）**
- **为什么这课能绕过 CFG**

回顾上课被拦的原因：

Code Cave 注入：

- 写入的是 .text 节末尾的 padding 区（0x00 或 0xCC）。
- 这个地址不是任何函数入口。
- CFG 在创建远程线程时，检查“起始地址是否在 CFG 合法目标表里”。
- 因为 padding 区不在表里 → 拒绝。

Module Stomping：

- 写入的是 AmsiScanBuffer 的函数开头。
- 这个地址在 amsi.dll 的导出表里，是 CFG 认可的合法函数入口。
- CFG 检查 → 放行。

结论：同样的 WriteProcessMemory + CreateRemoteThread，写在 padding 区被拦，写在导出函数入口就通过。这就是 Module Stomping 的核心价值。

- **对比实验：两版差异**

检查项	第一版（明文 API）	第二版（无 IAT）
IAT 里有 OpenProcess 等	有	无
二进制里有 "amsi.dll" 字符串	有	无
二进制里有 "AmsiScanBuffer" 字符串	有	无
目标进程必须加载 amsi.dll	是	是

第二版优势：静态扫描看不到敏感字符串，EDR 静态检测基本失效。

- **本课小结**

- Module Stomping 的本质：不找空隙，而是覆写一个合法导出函数。
- 关键区别：
  - Code Cave 写 padding（非法入口）→ CFG 拦；
  - Module Stomping 写导出函数（合法入口）→ CFG 放行。
- 不需要 VirtualAllocEx、VirtualProtectEx，因为 .text 节天然可执行。
- 靶子选择：amsi.dll!AmsiScanBuffer（PowerShell 必加载）。
- 无 IAT 版本：用封装的 GetProcAddressEx + 自研哈希，把明文全部消灭。